

LightLLM 项目技术原理与深度思考

从零构建的 C++/CUDA LLM 推理引擎技术解析

技术分析报告

2026 年 8 月 15 日

代码规模：95 源文件 · 约 23,000 行 · 19 测试 · 约 202,000 断言

目标平台：RTX 2060 6GB (Windows, CUDA 13.2) / RTX 4090 24GB (Linux, CUDA 11.8)

验证模型：Qwen2.5 0.5B / 1.5B / 3B / 7B

目录

1 项目总览：它是什么	4
1.1 一句话定位	4
1.2 实现的功能清单	4
1.3 实测性能概览	4
1.4 与主流引擎的关系	5
2 项目总体思维导图	5
3 系统架构	7
3.1 分层结构	7
3.2 一次请求的完整生命周期	7
3.3 关键设计决策	7
4 Tensor 抽象与内存管理	8
4.1 设计	8
4.2 悬垂指针 Bug: view 自赋值的教训	8
4.3 深度思考	8
5 GEMM：五级后端的渐进优化	9
5.1 各内核原理	9
5.2 深度思考	9
6 PagedAttention：分页 KV 缓存与解码内核	10
6.1 动机：KV 缓存的碎片化问题	10
6.2 Decode 内核结构	10
6.3 三项优化	11
6.4 深度思考	11
7 Paged Prefill Attention：分页上的长 prompt 处理	12
7.1 问题	12
7.2 批量首轮 prefill	12
7.3 深度思考	12
8 前缀缓存：Hash 与 Radix Tree	13
8.1 动机	13
8.2 Hash 模式（vLLM 风格）	13
8.3 Radix 模式（SGLang 风格）	13
8.4 深度思考	14
9 Continuous Batching 与调度器	14
9.1 问题	14
9.2 三种调度策略	14
9.3 深度思考	15

10 EngineServer: 异构批次执行引擎	15
10.1 step() 的完整流程	15
10.2 异构批次的数值一致性纪律	16
10.3 深度思考	16
11 采样系统	16
11.1 流水线	16
11.2 深度思考	17
12 fp16 混合精度	17
12.1 策略	17
12.2 深度思考	18
13 投机解码 (Speculative Decoding)	18
13.1 原理	18
13.2 项目实施	19
13.3 实测结论	19
13.4 深度思考	19
14 结构化输出 (xgrammar 集成)	19
14.1 原理	19
14.2 深度思考	20
15 Tokenizer 与模型加载	20
15.1 Tokenizer (GPT-2 BPE)	20
15.2 模型加载 (Safetensors)	21
16 测试与验证体系	21
16.1 规模与分层	21
16.2 方法论	21
17 关键 Bug 案例分析	21
18 性能数据	22
19 深度思考: 推理引擎的技术哲学	22
19.1 一切性能问题的总纲: Roofline	22
19.2 数值一致性是推理引擎的隐形地基	23
19.3 显存、算力、延迟的不可能三角	23
19.4 从零构建的价值	23
19.5 未来方向	23
A 项目文件结构速览	23
B 关键常量与配置	24

1 项目总览：它是什么

1.1 一句话定位

LightLLM 是一个从零构建的 C++/CUDA 大语言模型推理引擎，对标 vLLM 与 SGLang 的核心能力集：PagedAttention 分页 KV 缓存、前缀缓存、Continuous Batching（连续批处理）、采样系统、结构化输出与投机解码。整个项目不依赖 PyTorch/TensorFlow，仅依赖 CUDA 运行时、cuBLAS 与 xgrammar（结构化输出），所有算子（GEMM、注意力、归一化、RoPE 等）均为手写 CUDA kernel。

1.2 实现的功能清单

- **12 类手写 CUDA 算子**：GEMM（5 级后端）、PagedAttention（decode 3 变体 + prefill 2 变体）、RMSNorm、RoPE、Softmax、逐元素运算、7 种激活函数、5 种一元运算、4 种归约 + top-k、Embedding gather、lm_head。
- **PagedAttention**：以 16 token 为物理块的分页 KV 缓存；decode 内核融合 Float4 向量化、共享内存双缓冲、Split-K 三种优化；prefill 支持直接内核与 FlashAttention 式分块内核，直接从分页块读取 KV，无需重建连续张量。
- **前缀缓存**：Hash 前缀缓存（vLLM 风格）与 Radix Tree 前缀缓存（SGLang 风格）双实现，运行时可用环境变量 LIGHTLLM_PREFIX_CACHE 切换；带引用计数与 LRU 淘汰。
- **Continuous Batching**：FCFS / PrefillFirst / DecodeFirst 三种调度策略；长 prompt 分块 prefill 与 decode 混批；KV 容量预过滤实现优雅背压。
- **采样系统**：temperature、top-k、top-p、min-p、multinomial 采样与语法掩码。
- **fp16 混合精度**：权重 fp16 + Tensor Core GEMM（fp32 累加），显存减半；注意力与归一化保持在 fp32，保证数值安全。
- **结构化输出**：集成 xgrammar，支持 JSON Schema 与正则约束解码（bitmask 方式，与采样协同）。
- **投机解码**：自投机（目标模型前 M 层作草稿）与双模型两种路径；严格 argmax 接受与 Leviathan 概率接受；verify-as-decode 单次前向优化。
- **工程外围**：GPT-2 BPE tokenizer、safetensors（含分片）加载器、极小 HTTP server、跨平台 CMake 构建（Windows MSVC + Linux GCC）、19 个测试程序（约 202k 断言）。

1.3 实测性能概览

平台	场景	指标	数值
RTX 2060 6GB	单用户（Qwen2.5-0.5B fp32）	吞吐	15.8–18.7 tok/s
RTX 2060 6GB	3 用户 Continuous Batching	吞吐	25.1–25.3 tok/s
RTX 2060 6GB	手写 fp32 GEMM（Vectorized）	算力	1646 GFLOPS（cuBLAS 的 58%）
RTX 2060 6GB	fp16 Tensor Core GEMM	算力	~4.7 TFLOPS
RTX 4090 24GB	3 用户 Continuous Batching	吞吐	103.7 tok/s（2060 的 4.1×）
RTX 4090 24GB	并发上限（修复悬垂指针后）	并发	512 稳定（原 32）
RTX 4090 24GB	P50 TTFT / P50 TPOT	延迟	879 / 385 ms

1.4 与主流引擎的关系

项目明确借鉴了三个开源系统的核心思想：vLLM 的 PagedAttention 与 Continuous Batching、SGLang 的 RadixAttention 前缀缓存、FlashAttention 的在线 softmax 分块算法。但实现全部独立完成，且刻意保持”小而完整”：单机、单卡、单进程，聚焦在 6GB 消费级显卡上把整条推理链路跑通、跑对、跑快。这使它成为一个极佳的”推理引擎解剖标本”——本文档随后各章将逐一拆解其技术原理，并给出对每项技术的深度思考。

2 项目总体思维导图

在深入细节之前，先用一张思维导图总览整个项目的知识地图：中心是引擎本体，十个分支分别对应本文档的十章主题，每个分支再挂出该主题的关键技术点。阅读时可以先从这张图建立整体框架，再进入后续章节逐一展开。

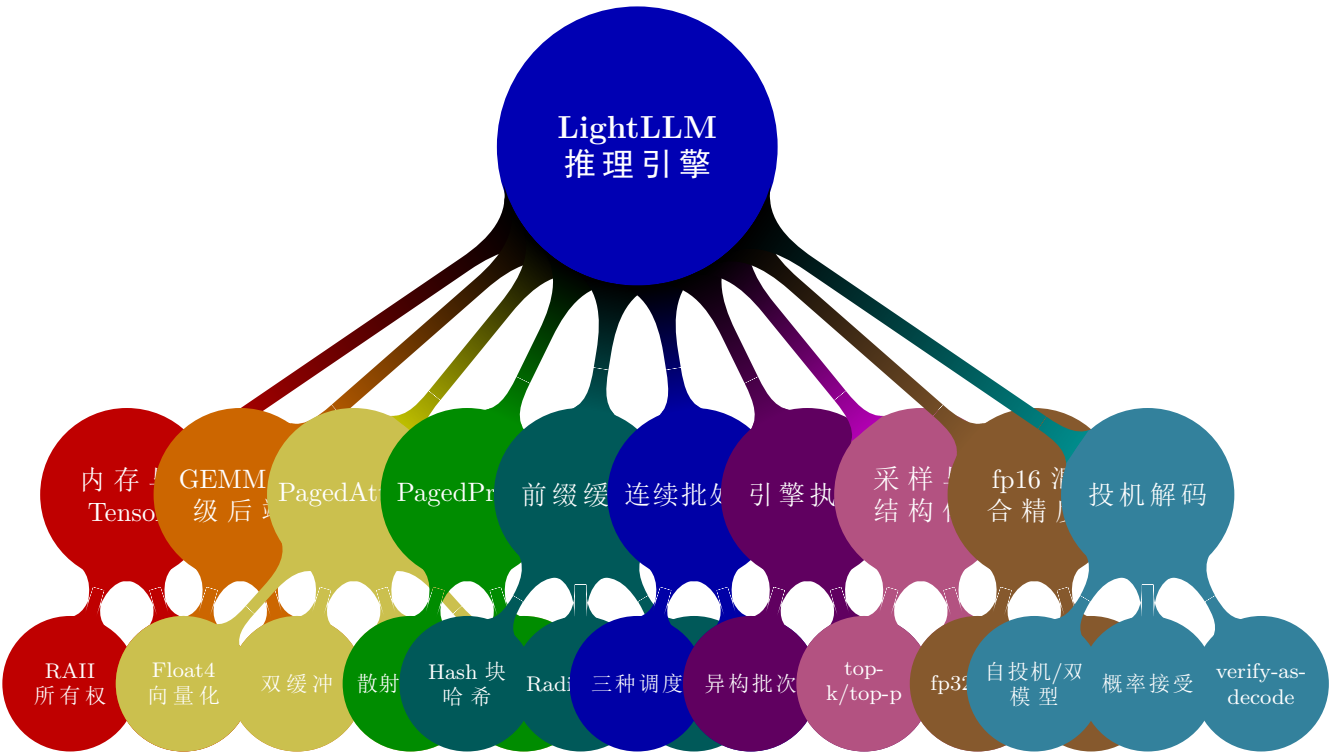


图 1: LightLLM 项目总体思维导图：十个技术分支与关键点

3 系统架构

3.1 分层结构

引擎采用清晰的分层设计，从下到上依次为：

```
+-----+
| HTTP Server / tools / tests          (最薄的外壳)          |
+-----+
| BatchMainLoop  调度-执行主循环: submit / step / metrics    |
|   |-- Scheduler    WAITING -> DECODING -> FINISHED 状态机  |
|   `-- EngineServer 异构批次执行: step() / speculate()      |
+-----+
| Model: ModelConfig + SafetensorsLoader + EngineLayerW 权重  |
| KV Cache: BlockAllocator + BlockTable + PrefixCache(抽象)   |
|           |-- HashPrefixCache  `-- RadixPrefixCache         |
+-----+
| Ops: gemm / norm / rope / attention / sampling / lm_head /  |
|       elementwise / activation / reduce / embedding / argmax |
| Tensor + CudaAllocator      (RAII 张量与显存分配)           |
+-----+
| CUDA Runtime + cuBLAS + xgrammar      (仅有的外部依赖)     |
+-----+
```

3.2 一次请求的完整生命周期

1. `BatchMainLoop::submit()` 创建调度器侧的 `Request` 与引擎侧的 `RequestState`，请求进入 `WAITING`；
2. `Scheduler::step()` 按策略产出一个异构批次 (`ScheduleStep`)：若干 `prefill` 分块（每块 $\leq \text{chunk_size}$ token）+ 若干 `decode` token（各 1 token）；
3. `EngineServer::step()` 执行整个批次的一层一层前向：embedding、RMSNorm、QKV 投影、RoPE、注意力（首次 `prefill` 走批量连续注意力；续块走分页 `prefill` 注意力；`decode` 走分页注意力）、O 投影、残差、SwiGLU MLP、残差；
4. `prefill` 条目更新 KV 缓存与计数；`decode` 条目经 `lm_head` + 采样产出 token；
5. 首个 `decode` token 记录 TTFT；EOS 或达到 `max_new_tokens` 时请求进入 `FINISHED`，释放其全部 KV 块；
6. 若开启投机解码，`decode` 条目由 `speculate()` 接管：草稿-验证-接受循环。

3.3 关键设计决策

- **单一前向实现**：历史上曾存在两套前向（`EngineServer` 与旧 `InferenceEngine`），并行维护导致 5 个 bug 只修了一半；最终删除旧实现，`EngineServer` 成为唯一前向路径。单一实现是数值一致性的前提。
- **层间 lockstep 分配**：每层一个 `BlockAllocator`，但所有层的分配/释放严格同步（同一物理块号），使 `BlockTable` 可以跨层共享同一份逻辑 → 物理映射，简化了生命周期管理。

- **主机侧元数据、设备侧数据**：block table、seq len 等小元数据在 CPU 侧维护，每步打包上传；K/V 与激活常驻显存。

4 Tensor 抽象与内存管理

4.1 设计

Tensor 是引擎的”细胞”：持有形状、dtype (F32/F16/BF16/I32)、设备 (CPU/CUDA) 与裸指针。它**移动语义**、**禁止拷贝** (unique ownership)，析构时自动释放 (RAII)。`view()` 返回**不拥有数据**的视图；`reshape_inplace()` 原地改形状。

4.2 悬垂指针 Bug: view 自赋值的教训

早期代码中广泛存在 `t = t.view(...)` 的写法。其执行序列是：

1. `t.view(...)` 构造一个临时视图 `v` (`owns_data=false`，但 `data_` 指向 `t` 的缓冲区)；
2. 移动赋值 `t = v`：先 `deallocate()` 释放 `t` 的缓冲区 (`cudaFree`)！
3. 再把 `v` 的 `data_` (已释放的悬垂指针) 移入 `t`；
4. 临时 `v` 析构 (不释放，因不拥有)，但 `t` 现在持有悬垂指针，后续任何使用或析构都会触发 `use-after-free` / `double-free`。

形象类比 把 Tensor 想成一张**借书证**，`cudaMalloc` 的缓冲区是图书馆的书。`view()` 只是”再复印一张借书证” (不拥有书，但指向同一本书)；而 `t = t.view(...)` 的悲剧在于：先把自己手里的书**还掉** (`cudaFree`)，再把复印证上的书号抄回来——于是手里的证指向一本已经还回书库的书，下次”取书” (kernel 访问) 就可能拿到别人的书甚至坠入虚空 (`use-after-free`)。修复 `reshape_inplace()` 相当于规定：”改借书证上的书名可以，但严禁先还书”。

后果：并发从 32 上升到 512 时频繁崩溃；`compute-sanitizer` 定位到 `rope_kernel` 越界 (错误计数 1124 → 0)。修复方式是新增 `reshape_inplace()`，所有”原地重解释形状”的调用点改用它；并发上限随之从 32 提升到 512。

4.3 深度思考

- **视图语义与所有权必须显式分离**。Python/PyTorch 靠引用计数掩盖了这类问题，C++ 中”返回非拥有视图 + 移动赋值”的组合天然危险。工程上要么让视图类型与拥有类型分离 (如 `std::string_view` 之于 `std::string`)，要么禁止在拥有者上赋值视图 (本项目的做法：提供原地 `reshape`)。
- **并发是内存 bug 的放大器**：单请求路径下悬垂指针可能”碰巧”不炸 (缓冲区未被复用)，高并发下内存布局变化使错误显性化。这也解释了为什么压测上限从 32 到 512 的提升本质上是**正确性修复的副产品**，而非性能优化。
- **compute-sanitizer 的定位思路**：先怀疑所有权，再怀疑越界——两类问题在 CUDA 下表现高度相似 (`illegal memory access`)。

5 GEMM：五级后端的渐进优化

GEMM ($C_{M \times N} = A_{M \times K} \cdot B_{K \times N}$) 是 Transformer 的计算主体。项目实现了 4 个手写 fp32 内核 + 1 个 cuBLAS fp16 Tensor Core 路径，可用 `set_gemm_backend()` 切换，形成一条教科书式的优化阶梯：

后端	分块	向量化	流水	实测 (RTX 2060)
Naive	无 (1 线程 1 元素)	无	无	430 GFLOPS
Tiled	16×16 , TK=8	无	无	481 GFLOPS
Vectorized	64×64 , 8×8 线程	float4	无	1646 GFLOPS
DoubleBuf	64×64	float4	乒乓缓冲	1502 GFLOPS
cuBLAS fp32	自适应	自适应	自适应	2833 GFLOPS
cuBLAS fp16	Tensor Core	—	—	~4700 GFLOPS

5.1 各内核原理

Naive: 每个线程算一个输出元素，内层循环直接读全局内存。A 的行元素在相邻线程间完全不复用，B 的列亦然，访存效率极低。

Tiled: 引入共享内存分块 (16×16 输出 tile, TK=8 的 K 分块)。每次迭代把 A 块与 B 块协作加载进共享内存，块内复用；这是“分块 (tiling)”思想的第一次落地，但 tile 小、无向量化，且存在 bank conflict 隐患。

Vectorized: 将 tile 扩到 64×64 , 16×16 线程、每线程 4×4 累加器 (register blocking)；加载使用 128-bit 的 float4 指令 (一次取 4 个 float)，并把共享内存 tile 的每行填充 1 个元素的 padding (PAD=1) 以避免 bank conflict。算力从 481 跳到 1646 GFLOPS——这是向量化 + 寄存器累加 + 消除 bank conflict 三者叠加的收益。

DoubleBuf: 双缓冲 (乒乓) 共享内存 tile，期望“计算当前 tile 的同时预取下一 tile”。实测反而回落到 1502 GFLOPS。原因见下文的深度思考。

cuBLAS fp16: cublasGemmEx 以 CUDA_R_16F 输入、CUBLAS_COMPUTE_32F 计算类型调用，走 Tensor Core, fp16 输入、fp32 累加输出，约 4.7 TFLOPS——这是 fp16 混合精度模式的 GEMM 主干 (gemm_f16f32)。

形象类比 把 GEMM 想成超市理货：Naive 内核是“每个售货员负责一个货架格，每次都要跑到仓库（全局内存）取一件货”，来回跑腿（访存）远多于干活。Tiled 是“先把一批货搬上柜台（共享内存），柜台边卖边补”；Vectorized 更进一步“用推车（float4）一次搬四件、柜台按 4×4 格分区（寄存器累加），通道之间留缝防堵车（bank conflict padding）”。DoubleBuf 则是“两个柜台轮换，一个在卖、一个在补货”——可惜本实现里“换柜台”本身要把货再搬一遍（共享内存拷贝），K 短时补货收益抵不过搬货开销，于是 1646 掉回 1502 GFLOPS：优化不是越多越好，要和问题规模匹配。

5.2 深度思考

- **算术强度 (arithmetic intensity) 决定一切。** fp32 GEMM 在 RTX 2060 上理论算力约 6.5 TFLOPS、显存带宽约 336 GB/s，平衡点 (roofline 交点) 约为 19 FLOP/byte。Naive 内核每个输出元素要读 $2K$ 个输入，算术强度 $\approx \frac{2}{1+1} = 1$ FLOP/byte 量级，被带宽锁死；分块后 tile 内复用使算术强度提升到 $\frac{2 \cdot TM \cdot TN \cdot TK}{(TM + TN) \cdot TK}$ ， 64×64 tile 约为 32 FLOP/byte，越过平衡点，才开始逼近算力上限。优化的本质不是“少算”，而是“少读”。

- **为什么双缓冲在这里失败了。**标准双缓冲应通过指针交换实现零拷贝切换；但本实现受限于共享内存数组无法动态改名，采用了”计算完后把 A1 拷贝回 A0”的交换方式($As0[m][k]=As1[m][k]$ 循环)，引入了 $2\times$ 共享内存读写的额外开销。K=1024 时 tile 迭代次数少 ($1024/8 = 128$ 次)，预取收益被拷贝开销抵消。这给出了一条可迁移的经验：**优化手段的收益是问题规模相关的**——流水线深度、分块大小都要与 K、显存带宽、共享内存带宽的比值匹配，盲目叠加优化可能适得其反。
- **为什么手写 fp32 追不上 cuBLAS：**cuBLAS 还有寄存器级 swizzle、双精度累加调度、L2 亲和性、多 kernel 分派等数十项微优化，58% 的差距是”正确方向上的合理收敛点”。但手写内核的价值在于**教学与可控性**，且 fp16 Tensor Core 路径直接复用 cuBLAS，把工程精力留给注意力等 cuBLAS 不覆盖的算子。

6 PagedAttention: 分页 KV 缓存与解码内核

6.1 动机: KV 缓存的碎片化问题

自回归解码中，每个序列的 K/V 张量随生成不断增长。若为每个序列预分配”最大长度”的连续显存，则显存利用率极低（实际长度通常远小于上限），且无法支持大量并发序列。vLLM 的 PagedAttention 借鉴操作系统分页思想：把 KV 缓存切成固定大小的物理块（本项目 BLOCK_SIZE=16 token），每个序列持有一张**逻辑** → **物理块表**（BlockTable），物理块按需分配、可被多个序列共享（前缀共享的基础）。

KV 存储布局: [num_blocks, 16, num_kv_heads, head_dim], K 与 V 各一整块连续池, kernel 通过 phys_id 直接索引。

形象类比 把 KV 缓存想成一座**图书馆**：每个物理块是书架上固定大小的一个格子（16 页），每个序列是正在写书的人，BlockTable 是他手里的**取书单**——”第 3 格放我笔记的第 1-16 页，第 7 格放第 17-32 页……”。生成新 token 时，注意力只需按取书单去对应格子取页（kernel 按 phys_id 索引），而不是把整本书的散页重新誊抄成一份连续手稿（旧方案的 cudaMemcpy 拼张量）。多位作者写相同的前言时还能**共用同一个格子**（前缀缓存共享），谁写完谁划掉借阅记录（ref_count 归零）把格子还回去。

6.2 Decode 内核结构

每个线程块处理一个 (token, q_head) 对：把该 token 的 Q 载入共享内存，然后遍历其块表中的每一个物理块，将块的 K/V tile 协作载入共享内存，对块内 16 个位置逐一做点积 + 在线 softmax 更新，最后归一化写出。核心循环：

Listing 1: 在线 softmax 的 rescale 更新（伪代码）

```

1 float m = -1e38f, l = 0.0f, acc = 0.0f;
2 for (每个 KV 位置 s) {
3     float dot = scale * dot(q, k_s);
4     float m_old = m;
5     m = fmaxf(m_old, dot); // 新 running max
6     float corr = expf(m_old - m); // 旧状态缩放因子
7     float term = expf(dot - m); // 新项
8     l = l * corr + term; // 归一化常数
9     acc = acc * corr + term * v_s; // 加权和
10 }
11 out = acc / (l + 1e-8f);

```

其数学依据是：若已知前 t 项的 $\ell_t = \sum_{i \leq t} e^{s_i - m_t}$ 与 $\text{acc}_t = \sum_{i \leq t} e^{s_i - m_t} v_i$ ，加入新得分 s_{t+1} 后， $m_{t+1} = \max(m_t, s_{t+1})$ ，则

$$\ell_{t+1} = \ell_t e^{m_t - m_{t+1}} + e^{s_{t+1} - m_{t+1}}, \quad \text{acc}_{t+1} = \text{acc}_t e^{m_t - m_{t+1}} + e^{s_{t+1} - m_{t+1}} v_{t+1}. \quad (1)$$

这避免了”先算完整 $\text{QK}^\top$ 再 softmax”的 $O(S^2)$ 显存占用，是 FlashAttention 系列算法的核心，也是分页解码能流式处理任意长 KV 的原因。

具体算例：设某头依次得到三个得分 $s = [2, 1, 3]$ （已乘 $\sqrt{d}$ 倒数），跟踪 m 与 ℓ 的演化：

步骤	新得分	更新过程	状态 (m, ℓ)
1	$s = 2$	首项, $\ell = e^{2-2} = 1$	$(2, 1.000)$
2	$s = 1$	max 不变: $\ell += e^{1-2} = 0.368$	$(2, 1.368)$
3	$s = 3$	max 更新: $\ell = 1.368 e^{2-3} + e^{3-3}$	$(3, 1.503)$

最终 $\ell = 1.503 = e^{2-3} + e^{1-3} + e^{3-3}$ ，恰是”以最大得分 3 为基准”的归一化分母；输出 $= \text{acc}/\ell$ 与标准 softmax 逐位一致。注意第 3 步**旧项整体缩小**（乘 e^{2-3} ）而非重算——这就是”在线”二字的含义：无论序列多长，状态只有两个数 (m, ℓ, acc) ，且永不回头。若序列继续加长，唯一的新开销是 $O(1)$ 的 rescale，而不是重扫全序列。

6.3 三项优化

(A) Float4 向量化：K/V 以 float4（128-bit）加载，单次访存取 4 个 float（fp16 下取 8 个 half），全局内存带宽利用率提升约 4×。要求 $D \bmod 4 = 0$ （fp16 要求 $D \bmod 8 = 0$ ），不满足时回退标量基线。

(B) 共享内存双缓冲：乒乓两块 K/V tile，计算第 b 块时预取第 $b+1$ 块，隐藏全局内存延迟。当 $D > 128$ 时双缓冲所需共享内存超过 48 KB（ $D + 4 \cdot 16 \cdot D$ 个 float），自动回退单缓冲。这是针对解码访存密集特性的正确取舍——与 GEMM 场景不同，此处双缓冲确实生效。

(C) Split-K：当实际序列长度 > 256 时，把 KV 块均分到 4 个 grid 维度并行计算，每个 split 输出部分 softmax 状态 (acc, ℓ, m) ，再由归约内核按公式 (1) 的结合律合并：

$$m = \max(m_A, m_B), \quad \ell = \ell_A e^{m_A - m} + \ell_B e^{m_B - m}, \quad \text{acc} = \text{acc}_A e^{m_A - m} + \text{acc}_B e^{m_B - m}. \quad (2)$$

空 split（无工作）写出单位元 $(\text{acc} = 0, \ell = 0, m = -\infty)$ ，归约时跳过（ $\ell = 0$ 即跳过），避免 $\infty - \infty$ 产生 NaN。

内核选择完全由运行时实测决定：host 端先拷回 seq_lens 求出真实最大序列长度，再在”基线 / V2 / Split-K”间分派——避免为短序列付出长序列优化（Split-K 归约、双缓冲 prologue）的固定开销。

6.4 深度思考

- **PagedAttention 的本质是”按需分页 + 流式 softmax”两个思想的组合：**前者解决显存碎片与共享，后者解决任意长上下文的数值稳定性与显存占用。二者缺一，分页就无从谈起（连续 KV 无法按块流式处理）。
- **解码是带宽问题，不是算力问题：**每个 token 的 $\text{QK}^\top$ 只有 $D \cdot S$ 次乘加，但 K/V 读取量是 $S \cdot D$ 字节；在 6GB 消费卡上，权重加载与 KV 读取共同构成瓶颈。因此 decode 内核优化的第一优先级永远是**减少/隐藏访存**（向量化 $\rightarrow$ 双缓冲 $\rightarrow$ 并行化），而非减少计算。

- **阈值与回退是工程成熟度的标志**：对齐检查、48 KB 共享内存检查、> 256 才 Split-K、 D 分档选择 B_r ——每个分支都对应一种真实硬件约束。从”一个 kernel 打天下”到”自适应分派”，正是推理引擎与玩具实现的差距。
- **GQA 的天然融合**： $\text{kv_head} = \text{q_head} / (\text{Hq}/\text{Hkv})$ 的整除映射让多 Q 头共享同一 KV 头，kernel 里只需一次索引换算，无需任何 repeat 操作——分页结构使 GQA 的显存收益（7B 模型 KV 减 7 倍）零成本兑现。

7 Paged Prefill Attention: 分页上的长 prompt 处理

7.1 问题

prefill 阶段一个请求要一次处理 P 个 token。最朴素的做法是先把历史 KV 从分页块逐 token `cudaMemcpy` 拼成连续张量再做标准注意力——但每 token 一次 D2D 拷贝既慢又随并发数线性膨胀。本项目直接在分页块上做 prefill 注意力：

- **scatter kernel**：一个线程块负责一个 token，把连续 K/V 写入正确物理块的槽位（ $\text{pos}/16$ 定块、 $\text{pos}\%16$ 定槽），一次 launch 完成整块 chunk 的散写；另有批量版本（`scatter_prefill_kv_batched`）一次性处理 N 个请求的全部首轮 prefill。
- **direct kernel**（ $P \leq 64$ ）：每（chunk token, q_head ）一个块，与 decode 内核同构，遍历全部历史块 + 自身块（可选因果掩码）。
- **tiled Flash 变体**（ $P > 64$ ）：每块处理 B_r 个 Q 行（ B_r 按 head_dim 分档：64/48/32），K/V tile 被 B_r 个 Q 行复用，提升算术强度；同样支持 Float4 与双缓冲。

相比逐 token memcpy 方案，加速 6–47×，并消除了并发用户数的显式上限（此前每个并发用户都要在 CPU 侧做一遍拷贝循环）。

形象类比 把 prefill 想成**邮局分拣**：模型刚算出一整段 prompt 的 K/V，像一大摞写好的信，要按”收件人”（token 在序列中的位置）投进对应格子的信箱（物理块槽位）。旧方案是邮差一封一封地跑（每 token 一次 `cudaMemcpy`），信越多越慢；新方案是**分拣机一次倒进去**（scatter kernel 一次 launch 投完全部信）。而”批量首轮 prefill”更进一步：如果所有人的信箱都是空的（历史 KV 为空），干脆在分拣台上直接按地址读信（连续 K/V 批量注意力），连投递都省了。

7.2 批量首轮 prefill

当一批请求都处于**第一次** prefill（历史 KV 为空）时，K/V 还是连续的，项目专门实现了**批量连续注意力**（`first_prefill_attn_batched`）：一个 kernel 内通过 token 累计和（`token_cumsum`）定位每个全局行属于哪个请求、在该请求内的局部位置，直接以连续 K/V 计算（因果边界按各请求的绝对位置判断），输出直接写入 flat 缓冲。这避免了”先把连续 K/V 散写入分页块、再按块读回”的无谓往返。

7.3 深度思考

- **prefill 与 decode 的访存形态完全不同**：prefill 的 Q 有 P 行，K/V 可以被多行 Q 复用（高算术强度，接近计算受限）；decode 的 Q 只有 1 行，K/V 每行只被读一次（纯带宽受限）。因此 prefill 用”大 tile 复用”路线（Flash 分块），decode 用”极致访存”路线（向量化 + 双缓冲）——同一算子，两种物理。

- **分批 (batching) 优先于分页**：首轮 prefill 的 K/V 尚连续，批量连续注意力以 $O(N)$ 元数据换 N 次 kernel launch 的消除；只有当历史非空（续块）才退回分页路径。这是“数据布局决定算法选择”的又一例证。
- **共享内存超限的静默失败**：hd=128 时 first-prefill 内核动态共享内存超过 sm_75 默认 48 KB 上限，kernel 启动失败但不报错，输出全 0（详见 Bug 分析章）。教训：CUDA kernel launch 失败是静默的，任何“输出异常”都必须先怀疑 launch 配置，再怀疑算法。

8 前缀缓存：Hash 与 Radix Tree

8.1 动机

多用户场景（如多轮对话、共享系统提示词）中大量请求共享相同前缀。前缀缓存让新请求直接复用已计算过的 KV 块，省去重复 prefill。项目实现两个策略，构造时经 PrefixCachePolicy 选择（默认读环境变量 LIGHTLLM_PREFIX_CACHE）：

形象类比 把 prefill 想成搭乐高：每个 token 的 K/V 是一块积木，一条 prompt 是一条拼好的长龙。多轮对话的第二轮 prompt = 第一轮 + 新问题，如果每次都要从头拼一遍，纯属浪费。前缀缓存 = 把拼好的龙头存放在展示架上（缓存块），新请求先查“这段龙头我有没有拼过”，有就直接搬来用（共享块），只需补拼新尾巴。Hash 模式像给每 16 块一组的积木拍照片编号（FNV 哈希），按编号找整组；Radix 模式像按形状搭的索引树，能精确找到任意长度的公共段——甚至可以把一条拼好的龙从中间拆开（分裂），把新分支插进去，拆下的旧段照常供其他请求使用。展架位置有限时，最久没人碰的龙头会被清走（LRU 淘汰）。

8.2 Hash 模式（vLLM 风格）

- 每个完整物理块（16 token）独立计算 FNV-1a 64 位哈希，存入 `unordered_map<hash, block_id>`；
- 新请求 prefill 时逐块哈希查找，命中则 `ref_count++` 共享，未命中则分配新块；查找按块边界推进，遇首个未命中即停止（只匹配连续前缀）；
- LRU 淘汰：`ref_count==0` 且最久未访问者优先驱逐；注意 `remove()` 故意不刷新时间戳——淘汰序必须反映“最后一次被有用访问”，而非释放顺序。

优点： $O(1)$ 查找、实现简单。缺点：只能整块匹配（16 token 粒度），且哈希碰撞理论存在（工程上可忽略）。

8.3 Radix 模式（SGLang 风格）

Patricia trie（压缩前缀树）以变长 token 段存储路径：叶子节点持有完整 16-token 段与物理块号；内部节点是纯路由（`block_id=-1`）。子节点以段的首 token 为键，保证 $O(\log \text{fanout})$ 的查找。核心操作：

- **lookup**：从根沿 token 序列走，能整段匹配的叶子才记录块号；停在第一个失配处。由于树编码了绝对位置（根到叶的路径即序列前缀），查找必须从位置 0 开始，先走到 `start_pos` 再开始记录；
- **insert / split**：新块插入时若与现存叶子部分匹配，则在分叉点分裂该叶子：公共前缀提升为内部节点，原叶子与新生叶子成为其子节点。分裂不改变其他序列看到的根到叶路径，因此即使叶子 `ref_count>1` 也可安全分裂；

- **evict**: 后序遍历找 `ref_count==0` 且 `last_access` 最老的叶子;
- **remove**: 只把 `ref_count` 置 0, 不删除节点——节点作为路由骨架保留, 供未来查找, 物理块由 `evict` 真正回收 (`prune_leaf` 在父节点只剩单子时合并, 维持 `radix` 压缩性质)。

优点: 跨块、任意长度的前缀匹配 (最长公共前缀), 匹配粒度从”块”提升到”token 路径”; 共享检测不需要调用方逐块哈希探测。代价: 实现复杂度显著更高 (分裂/合并/父指针维护)。

8.4 深度思考

- 前缀缓存是用显存换算力: 缓存块占用物理块, 而物理块是并发数的硬约束。因此 `ref_count` 生命周期与 LRU 淘汰不是优化, 而是正确性要求——没有它们, 缓存会把并发容量吃光。
- **Hash 与 Radix 的取舍是”匹配精度 vs 结构复杂度”**: Hash 适合 prompt 前缀恰好对齐 16 token 边界的场景 (多数系统提示词场景), Radix 在任意长度共享时更优。工程上两者可共存 (vLLM 近期也在块粒度上叠加了更多结构), 本项目的抽象接口 (`PrefixCache`) 使策略可插拔, 是好的架构决策。
- 惰性删除 (`remove` 不删节点) 是 `radix` 树工程化中最精妙的一点: 删除立即释放物理块, 但保留路由骨架, 使”已淘汰路径”仍可被查找 (虽然匹配不到叶子) 并自然引导后续插入——避免了频繁分裂/合并的抖动。
- 前缀缓存与投机解码的潜在冲突: 投机验证写入的 KV 若进入共享块, 会污染其他序列的前缀 (本项目通过”目标缓存只允许 batched 权威路径写入”规避)。这提示: 缓存共享的前提是写入方单一且权威。

9 Continuous Batching 与调度器

9.1 问题

朴素批处理 (静态 batching) 要求所有请求同进同出: 先等一批请求凑齐, `prefill` 全部, 再逐个 `decode` 到全部结束。缺点是: 先到的请求要等后来者 (TTFT 恶化); `decode` 阶段吞吐随请求陆续结束而塌陷; 长 prompt 请求会拖垮整批。

Continuous Batching 把”批”从请求粒度细化到 **token** 粒度: 每步 (iteration) 由调度器自由组合任意请求的任意数量的 token (`prefill` 分块或 `decode token`) 成一个异构批次, GPU 每步都满载。

形象类比 静态批处理像**食堂包餐**: 必须等一整桌客人到齐 (一批请求凑够) 才开火, 大家一起吃完一起走——先到的人饿着等, 后到的人吃不到热菜, 一桌人里有人细嚼慢咽 (长 prompt) 其他人就得陪着。Continuous Batching 像**自助餐流水线**: 厨师 (GPU) 的每一勺 (每步 batch) 都盛得满满当当, 但**谁来了就给谁盛一勺**——先到先吃 (`prefill` 分块), 正在吃的人优先续盘 (`DecodeFirst`: `decode token` 先占预算), 新人见缝插针。于是厨师永远不闲着, 客人也永远不用等整桌人。分块 `prefill` 则是”把一大份菜 (长 prompt) 切成小份分批上”, 避免一个人占着厨师十分钟。

9.2 三种调度策略

- **FCFS**: 一次只处理一个请求 (先 `prefill` 完再 `decode`), 最简单, 但无并发性——仅作参考实现。

- **PrefillFirst:** prefill 优先（每步最多一个请求一个 chunk），decode token 随后全部收集。prefill 延迟最低，但 decode 用户可能被新请求抢占。
- **DecodeFirst（默认）:** decode token 全部优先占用 token 预算（`max_batch_tokens=256`），剩余预算才用于 prefill 分块（每块 $\leq \text{chunk_size}=16$ ，且每步最多 `max_prefill_entries` 个 prefill 条目）。交互用户（正在生成）不被新请求打断，长 prompt 通过分块 prefill 在多个步骤间穿插执行。

请求状态机: `WAITING` $\rightarrow$ (`prefill 分块  $\times n$`) $\rightarrow$ `DECODING` $\rightarrow$ `FINISHED`。每个 `ScheduleEntry` 记录 `request_idx / num_tokens / start_pos / is_prefill`，`start_pos` 是 RoPE 需要的绝对位置。

9.3 深度思考

- **调度本质是 SLA 决策:** 同一份 GPU 资源，PrefillFirst 优化 TTFT、DecodeFirst 优化 TPOT 稳定性（不让在跑的请求变慢）、FCFS 优化实现简单性。没有“最优”策略，只有与业务目标匹配的策略——这也是工业引擎把调度策略做成可配置插件的原因。
- **token 预算（256）是吞吐与延迟之间的旋钮:** 预算越大，单步 batch 越大，GEMM 效率越高，但每步耗时变长、TTFT 变差；预算越小，交互越灵敏，但 GPU 可能喂不饱。256 恰好是“单步解码 + 少量 prefill”在消费卡上的经验甜点。
- **分块 prefill（chunked prefill）是 Continuous Batching 的隐藏基石:** 没有它，一个 8k prompt 会独占整批 512 个步骤之一，所有 decode 用户集体卡顿一个 prefill 时长；分块后长 prompt 的算力消耗被摊薄到多个步骤，与 decode 交错，以轻微的总吞吐损失换取数量级更好的尾延迟。
- **引擎侧的容量预过滤（backpressure）补上了调度的盲区:** 调度器不知道 KV 池水位，若放任分配会在池耗尽时崩溃。`EngineServer::step()` 前置过滤：先为 decode（在跑请求）预留块，再为 prefill（新请求）预留；不足的条目直接 deferred 到下一步。这是“调度与执行解耦后必须补的契约”。

10 EngineServer：异构批次执行引擎

10.1 step() 的完整流程

1. 容量预过滤：见上节；
2. 块分配：prefill 首次条目走前缀缓存（lookup $\rightarrow$ 共享块 `ref++` $\rightarrow$ 新块分配）；后续条目按累计 token 数补块；decode 在 16 边界跨块时分配；投机模式预分配 k 个候选位；
3. 输入构建：所有条目的 token id 拼成 flat 向量，一次 embedding gather 得到 $[T, D]$ 激活（避免每请求一次 kernel launch）；
4. 逐层前向（每层）：RMSNorm $\rightarrow$ QKV GEMM（+Qwen2 的 q/k/v bias） $\rightarrow$ reshape $\rightarrow$ RoPE（CPU 生成 cos/sin，decode 条目的绝对位置以 `RequestState.seq_len` 为准，而非调度器给的过期 `start_pos`） $\rightarrow$ 注意力 $\rightarrow$ dtype 转换 $\rightarrow$ O 投影 + 残差 $\rightarrow$ RMSNorm $\rightarrow$ SwiGLU MLP + 残差；
5. 注意力三支：首次 prefill 批量连续注意力；续块 prefill 分页注意力（逐请求）；decode 批量分页注意力（先写本 token 的 K/V 再注意力，使自注意力可见，与 causal 语义一致）；

6. 前缀缓存登记：本步写满的块登记入前缀缓存；
7. 后处理：捕获 pre-norm hidden（供投机使用）；final norm；prefill 完成后预采样首个 **decode token**（grammar 感知）；decode 条目 lm_head + 采样 + 更新状态。

形象类比 把一次 `step()` 想成交响乐团的一次排练：指挥（调度器）拿到今天的曲目单（`ScheduleStep`），上面既有新乐章的开头（prefill 分块）也有老乐章的结尾（decode token）。乐手（GPU）按谱子（逐层 kernel）从头到尾演奏一遍，但每个音符的“身份”都不同——有人是新谱第 3 小节（RoPE 位置 3），有人是老谱第 1,024 小节。任何“按行号猜身份”的偷懒都会奏错音（用错 `start_pos`、写读顺序颠倒），所以乐谱上必须显式标注每一行的绝对位置（entry 元数据）。容量预过滤则是“先看看乐器够不够（KV 块），不够的曲子这遍先不排，等上一首结束腾出乐器再排”。

10.2 异构批次的数值一致性纪律

代码注释反复强调几条纪律，值得原文转述：

”Writing after attention made the decode hidden states diverge from the model for `hd=128`（第二个及以后的生成 token 错误）。”

即 decode 的 K/V 必须先写后读；prefill 的 last-token hidden 必须捕获 **pre-norm** 版本（否则 lm_head 双重归一化）；投机验证必须复用与常规解码同一套 **batched kernel**（否则 KV 数值分叉，接受率塌缩）。

10.3 深度思考

- 异构批次的最大敌人是“隐式假设”：同一份 $[T, D]$ flat 张量里混着 prefill 行与 decode 行，任何“行间关系”（如 RoPE 位置、注意力边界）都必须显式从 entry 元数据推导。本项目两次栽跟头（decode 用错 `start_pos`、self-attention 写读顺序）都源于此。异构 = 每行携带自己的语义，*kernel* 必须逐行解释。
- 预采样首个 **decode token** 是一步聪明的融合：prefill 完成后最后一层的 hidden 本就要算，顺手经 lm_head 采样出第一个 token，把“prefill 结束 + decode 开始”两步压缩为一步，TTFT 直接省一次全模型前向。
- 元数据全部 **H2D 打包上传**：block table、seq_len、偏移、累计和等小数组每步一次批量拷贝，避免逐请求同步——在 PCIe 带宽面前，把 N 次小同步合并成 1 次大同步是通用原则。

11 采样系统

11.1 流水线

`sample()` 在 CPU 侧对 logits 依次执行：

1. 贪心短路：temperature < 1e-5 直接取（语法掩码内的）argmax；
2. 语法掩码：把掩码外 token 置 $-\infty$ （与采样正交的硬约束）；
3. 温度缩放： $l_i \leftarrow l_i/T$ ；

4. **top-k**: `partial_sort` 取前 k , 其余置 $-\infty$;
5. 数值稳定 **softmax**: 三趟 ($\max \rightarrow \exp\text{-sum} \rightarrow$ 归一化);
6. **top-p (nucleus)**: 按概率降序累加到 p 为止, 截断并重归一化;
7. **min-p**: 低于 $\max p \cdot \min_p$ 的置零并重归一化;
8. **multinomial**: CDF + `lower_bound` 二分查找。

形象类比 采样像转盘抽奖: logits 是每个奖项的”呼声”, softmax 把呼声换算成转盘上的扇形面积。temperature 是把扇形重新分配的旋钮——温度高 ($T > 1$) 时面积趋近均匀 (更随机), 温度低时最大扇形吞掉其余 (更确定), $T = 0$ 时干脆”永远只开最大那个扇区” (贪心)。top-k 是”只保留面积最大的 k 个扇区”; top-p 是”从大到小攒到面积占比 p 就封盘”; min-p 是”面积小于最大扇区 $\min_p$ 倍的直接作废”。语法掩码更狠: 直接把非法奖项从转盘上抠掉, 剩下的重新归一化再转。

每一步都有防御性 fallback (过滤后空集 $\rightarrow$ 原 logits argmax), 保证任何参数组合下采样器都不会崩溃或返回非法 token。

11.2 深度思考

- 为什么采样放在 CPU: 词表 15 万、每步每请求一次。CPU 单次采样约几十微秒, 在小并发 (≤ 512) 下 GPU 前向 (毫秒级) 完全掩盖之; 而 GPU 采样需要额外的 kernel launch 与 D2H 同步。这是延迟预算分配问题: 把 CPU 的空闲时间花在低延迟路径上, GPU 留给带宽敏感的计算。当并发与吞吐进一步放大时 (数千并发), CPU 采样会成为瓶颈, 届时才需要 GPU 采样器——实现里保留的 `sample_batch` 接口正是为此预留。
- 过滤顺序有讲究: top-k 在 logits 上做 (避免排序 15 万概率), top-p / min-p 在概率上做 (需要归一化后的相对量纲)。顺序错乱会导致语义偏差。
- 贪心也是”带掩码的贪心”: 语法约束下”贪心”必须定义为”合法集合内 argmax”, 这使贪心、采样、结构化输出三条路径共享同一份 logits 语义。

12 fp16 混合精度

12.1 策略

- 权重以 fp16 常驻显存 (BF16 权重加载时转 fp16), 显存减半 (0.5B 权重约 1 GB $\rightarrow$ 0.5 GB);
- GEMM 走 `cublasGemmEx` Tensor Core: fp16 输入、**fp32 累加** (`CUBLAS_COMPUTE_32F`), 输出 fp32;
- 注意力全程 fp32: Q/K/V (GEMM 输出即 fp32)、scores、softmax、输出; 只在 O 投影前把注意力输出 cast 回 fp16 (`cast_f32_to_f16` kernel);
- KV 缓存保持 fp32 (构造参数可改 `kv_dtype`);
- `lm_head` 有 fp16 快速路径 (h 与权重均 fp16 时直接 `gemm_f16f32`)。

形象类比 fp16 混合精度像商人的两本账：货单（权重）用“简写符号”（fp16）记，省纸（显存减半）；但每笔成交的总额、找零（累加和、softmax 分母）必须在计算器（fp32）上算，算完才把结果用简写记回账本。注意力像“对账”——每一笔都要精确核对（scores/softmax 全程 fp32），只在对完账之后、入账之前（O 投影前）才压缩成简写。为什么货单可以简写而账目不能？因为货单（权重）在训练时早就习惯了“被压缩着用”，而账目（中间统计量）一旦写错一位，错误会像滚雪球一样逐层放大。

12.2 深度思考

- 精度放在哪里的选择，本质是“错误传播敏感性”分析：累加器必须 fp32（Tensor Core 硬性要求，也是数值安全底线）；softmax 的 exp 对输入幅值极敏感，保持 fp32；RMSNorm 的均值/方差统计保持 fp32；而权重本身对低位噪声鲁棒（训练时已适应量化噪声），可以 fp16。权重可以降低精度，中间统计量不能——这是混合精度设计的通用判据。
- KV 缓存保持 fp32 是有意的保守：KV 会被反复读取并参与点积，其误差随序列长度累积；fp16 KV 在长上下文下会漂移（工业界近年才通过 KV 量化 + 补偿技术解决）。先正确、后压缩，是合理的工程次序。
- fp16 实测 4.7 TFLOPS 说明 Tensor Core 在消费级卡上受限于带宽与调度，但这条路径让 6GB 卡能跑 7B 模型（配合分片加载），是“显存容量 $\gg$ 峰值算力”场景下的正确取舍。

13 投机解码（Speculative Decoding）

13.1 原理

自回归解码每步只产生一个 token，却要加载全部权重（带宽受限）。投机解码用一个小（或浅）模型草稿 k 个候选 token，目标模型一次批量验证全部候选，从而一次权重加载产生多个 token，且不改变输出分布（数学等价）。

$$\text{接受率 } a = \sum_t p_d(t) \min\left(1, \frac{p_t(t)}{p_d(t)}\right) = 1 - \text{TV}(p_d, p_t), \quad \text{加速比} \approx \frac{1 + ak}{1 + kr + c} \quad (3)$$

其中 r 为草稿/目标权重比， c 为验证等固定开销，TV 为总变差距离。概率接受（Leviathan）：以 $\min(1, p_t(t)/p_d(t))$ 接受草稿 token，拒绝后从残差 $\max(0, p_t - p_d)$ 归一化采样修正 token——该方案保证发射分布恰为 p_t 。

形象类比 投机解码像学生做题、老师批改：每道题（每个 decode 步）都让老师（大模型）从头写一遍太慢——老师读题（加载权重）的时间远大于写字（计算）的时间。于是请一个反应快但水平低的小助手（草稿模型）一口气先写 k 个答案，老师拿标准答案（一次批量前向）一次性批改：从第一题起，对的留下（接受），第一个错的位置由老师自己写正确答案（修正 token），后面的答案全部作废（连续前缀约束，因为那些答案建立在错误答案之上）。小助手“押题”越准（ p_d 与 p_t 越接近），一次批改能留下的答案越多。奇妙的是：只要小助手是按自己的水平随机押题而非死记硬背，且老师按“押中概率比”来批改，那么最终交上去的卷子与老师亲笔写的在统计上完全等价——这就是“输出分布守恒”定理。

13.2 项目实施

- **DraftEngine 抽象: SelfSpecDraftEngine** (目标前 M 层作草稿, 共享权重但自有 KV 缓存; 同步时把目标权威 KV 逐块拷贝而非重算, 保证草稿上下文与目标逐位一致) 与 **DualModelDraftEngine** (独立小模型, 显存预算取”实际剩余显存”, 与目标共驻留);
- **verify-as-decode**: 把 decode 条目全权交给 **speculate()** (草稿 + 一次批量验证 + 接受循环), **prefill** 仍走 **step()**, 使投机请求每步只做 1 次目标前向 (原为 2 次: 常规 decode + 验证), 加速比从 $0.98\times$ 突破到 $1.04\times$;
- **接受循环**: 逐位置比对 (严格 **argmax** 或概率接受 + 语法掩码); 首个拒绝处采样修正 token 并用单 token 前向覆盖该位置 **K/V**; 已接受草稿推进语法状态机;
- **数值一致性**: 验证必须复用与常规解码同一套 **batched kernel (forward_batched)**, 否则 KV 分叉导致同模型草稿接受率也塌缩 (实测批量验证下 6.2% vs 顺序验证 93.8% ——GEMM 对批量尺寸数值敏感)。

13.3 实测结论

- 同模型自投机 ($M=24$): 接受率 93.8% , 输出与常规解码逐 token 相同 (KV 一致性的硬门禁);
- $0.5B \rightarrow 1.5B$ 双模型: 贪心草稿接受率 0% ; 采样草稿 + 概率接受升至 96% (与 $1 - TV$ 理论吻合); **verify-as-decode** 后加速比 $1.04\times$ 。

13.4 深度思考

- **草稿必须采样而非贪心**: 贪心草稿永远提出自己的 $\text{argmax } t^*$, 接受概率退化为 $p_t(t^*)$ (跨规模模型下可能 10^{-5}); 采样让草稿有机会提出目标也看好的 token ($p_t \geq p_d$ 者无条件接受)。这是本项目接受率 $0\% \rightarrow 96\%$ 的根本原因, 也是理论 ($a = 1 - TV$) 与工程 (采样器) 在代码里第一次真正交汇。
- **KV 缓存是投机解码的最大陷阱**: 草稿若写目标缓存, 验证时上下文就被污染; 验证若用不同的 kernel 路径, 写出的 KV 与常规解码不一致, 后续 token 全部偏离。权威缓存只允许一条写入路径是本项目最重要的不变量之一。
- **数值复现性高于直觉**: $M=1$ 与 $M=k$ 的 GEMM 浮点结果不同 (归约顺序差异), 使”同模型草稿必然 100% 接受”的直觉失效。这提醒我们: 在推理引擎里, 正确性的判据是”与权威实现的逐位一致”, 而非”看起来合理”。
- **自投机的局限**: layer-slice 草稿在 M 接近全模型时接受率悬崖式跳变 ($0.5B$ 上 $M=18$ 仍 0% , $M=24$ 为 93.8%) ——中间层表示不直接预测下一个 token, 最终决策集中在最后几层。真实加速依赖独立小模型或 EAGLE/Medusa 式草稿头。

14 结构化输出 (xgrammar 集成)

14.1 原理

结构化输出把”解码必须符合 JSON Schema / 正则”编译成文法状态机 (xgrammar 内部为上下文无关文法 + DFA 化)。每步解码时:

1. `CompileJSONSchema / CompileRegex` 编译请求级文法（带缓存）；
2. `FillNextTokenBitmask` 依据当前 FSM 状态输出合法 token 位图（vocab/32 个 int32，经 `DLTensor` 传递），引擎把它当采样掩码；
3. 采样只在掩码内进行（贪心 = 掩码内 argmax）；
4. `AcceptToken` 推进状态机；`IsTerminated` 检测文法完成（如 JSON 对象闭合）即结束生成；投机解码中被拒草稿用 `Rollback` 回滚 FSM。

形象类比 结构化输出像填答题卡：文法（JSON Schema）是答题卡上预先印好的空格与选项——”这里必须是数字”、”这里必须是”name”:"”。每走一步，答题卡（FSM 状态）告诉我们当前空格允许填哪些内容（bitmask 置位的合法 token），模型只能在允许的选项里选（贪心 = 选分最高的合法项）。投机解码的草稿则像”先用铅笔把整张卡填完再统一检查”：填到某一格发现不允许（语法拒绝），就把这一格之后的铅笔痕迹全部擦掉重填（Rollback 回滚 FSM + 修正 token），已擦掉的格子不影响后面（被拒草稿不推进状态机）。

与投机解码的协同：草稿 token 逐位 `AcceptToken` 校验，非法即拒；修正 token 在掩码内采样；已接受草稿推进 FSM。

14.2 深度思考

- 掩码是”硬约束”与”软采样”的解耦点：文法只决定哪些 token 合法，温度/top-k/top-p 决定在合法集内怎么选。二者正交，使结构化输出可以叠加在任何采样策略与投机解码之上——这是 xgrammar 的 bitmask 接口设计的高明之处，本项目对其的集成方式（掩码贯穿贪心、采样、投机三路径）完全复用了这个正交性。
- 投机 + 文法需要状态回滚：草稿是一次性批量提出的，验证时逐位推进 FSM，一旦拒绝就必须把 FSM 回滚到接受前缀处，否则状态机与真实序列脱节。任何”乐观执行”的优化都要配套”回滚协议”。

15 Tokenizer 与模型加载

15.1 Tokenizer（GPT-2 BPE）

- 手写 JSON 解析器读取 `tokenizer.json` 的 vocab 与 merges（不引入 JSON 库）；
- 字节级编码：GPT-2 风格 byte-to-unicode 映射（可打印字节直映，控制字节映射到 256+ 区段），保证任意 UTF-8 文本可无损 tokenize；
- **encode**：文本 → 逐字节 token → 反复扫描最高优先级相邻对并合并（HF 风格 greedy BPE；注释明确警告”单趟按 rank 顺序合并是错的”）；
- **decode**：token → 字符串 → 字节逆映射。

形象类比 BPE 分词像用乐高小人拼汉字：先把整段文字拆成最细的”字节颗粒”（每个字节一块小积木），再按一本优先级手册（merges 表）反复把相邻两块拼成更大的块：”手册上说‘Th’ + ‘e’ 优先拼成‘The’，那就先把所有‘Th e’都拼了，再拼下一档……直到没有可拼的相邻对”。字节级编码则保证任何文字（包括 emoji）都能拆成积木——先给每个字节编一个安全的字符代号（byte-to-unicode 映射），避免控制字符捣乱。本项目

C++ 版与 HuggingFace 版的差异（同一句英文切出 14 个 vs 5 个 token）就像两本手册的拼法略有不同：拼出的块形状不同，但拼出来的句子（解码文本）完全一样。

已知局限：C++ 实现的 BPE 切分与 HF 的 `tokenizers` 库存在差异（同一句“The capital of France is” C++ 切 14 token、HF 切 5 token），文本级正确但非最优——这是无第三方依赖的代价，已在文档中如实标注。

15.2 模型加载（Safetensors）

- 解析 8 字节头长 + JSON 头（dtype/shape/data_offsets），直接按偏移 `seek + read` 读张量字节；
- 支持单文件与分片（`index.json` 的 `weight_map` 收集 shard 列表）；
- BF16 权重在 CPU 侧转 fp32 或 fp16 后上传 GPU；
- 支持 `tie_word_embeddings=false`（如 7B）时独立加载 `lm_head`。

16 测试与验证体系

16.1 规模与分层

19 个测试程序、约 202k 断言、双平台（MSVC + CUDA 13.2 / GCC 11 + CUDA 11.8）零失败。测试覆盖：Tensor/算子正确性（与 CPU 参考对拍）、采样（14 万断言）、PagedAttention（基线/V2/Split-K 与朴素参考一致）、前缀缓存（176 断言，含 radix 分裂/淘汰）、引擎端到端（E2E 与 transformers 输出对齐）、fp16 路径、投机解码（输出恒等、接受率）、结构化输出、并发与混合长度仿真。

16.2 方法论

- 分层隔离 + 独立参考实现：transformers → 纯 numpy 逐层参考（float64）→ C++ 引擎 DEBUG dump，三层对拍。参考实现必须独立于引擎——若 numpy 参考复用了引擎的同一错误假设（如用 `embed.T` 当 `lm_head`），会“一致地错”；
- `compute-sanitizer` 定位非法内存访问（悬垂指针案）；
- 参数巧合掩盖 bug 的教训：0.5B（hd=64）碰巧躲过 smem 超限（hd=128 才触发）与 RoPE 线程映射 bug（位置 0 时 RoPE 恒等），导致小模型测试全绿而 1.5B/7B 全垃圾——多模型矩阵测试是正确性的必要条件。

17 关键 Bug 案例分析

Bug	根因	教训
q/k/v bias 缺失	Qwen2 保留投影 bias (LLaMA 去掉), 漏加后 错位	架构差异必须来自 config, 而非经验假设
RoPE 线程映射	每 (token, head) 只旋转了 1 个元素, 位置 0 恒等掩盖	位置 0 的”正确”会掩盖一切旋转 bug
双重归一化	pre-sample 从 post-norm hidden 再 norm 一次	pre/post-norm 语义必须显式标注
smem 超 48KB	kernel launch 静默失败, 输出全 0	launch 配置是排查第一站
untied lm_head	7B 有独立 lm_head, 引擎无条件用 embed	权重绑定方式要读 config
view 悬垂指针	自赋值释放自身缓冲区	视图/所有权分离; 并发放大内存 bug

18 性能数据

平台	场景	指标	数值
2060	单用户生成	吞吐	15.8–18.7 tok/s
2060	3 用户 CB	吞吐	25.1–25.3 tok/s
2060	32 并发	稳定性	稳定
2060	GEMM fp32 Vectorized	算力	1646 GFLOPS (cuBLAS 58%)
2060	GEMM fp32 DoubleBuf	算力	1502 GFLOPS (K=1024 swap 开销)
2060	GEMM fp16 Tensor Core	算力	~4.7 TFLOPS
4090	单用户	吞吐	22.9 tok/s
4090	3 用户 CB	吞吐	103.7 tok/s (4.1×)
4090	16 用户混合长度	吞吐	~40 tok/s
4090	P50 TTFT / TPOT	延迟	879 / 385 ms
4090	并发上限 (修复后)	并发	512 稳定 (原 32)

19 深度思考：推理引擎的技术哲学

在前文各节的”深度思考”之外，这里做一次全局性的综合。

19.1 一切性能问题的总纲：Roofline

推理引擎的每一个优化决策都可以放到 roofline 模型上理解：横轴是算术强度（FLOP/byte），斜线区是带宽受限，水平区是算力受限。**decode** 在带宽区，**prefill** 在算力区。于是：

形象类比 Roofline 像高速公路的通行能力：带宽是”每车道每秒过多少辆车”（字节/秒），算力是”每辆车能装多少货”（FLOP/秒）。一辆车装货太少（算术强度低），路再宽也白搭——堵在车流里，这是 decode：每次只运 1 个 token 的货，却要满载权重跑一趟，瓶颈是**车道数**（显存带宽）。车装得够满（算术强度高），瓶颈才变成”车本身开多快”——这是 prefill：大批 token 一起运，瓶颈是**发动机**（GPU 算力）。Continuous Batching 则是**拼车**：把多个请求的货装进同一辆车，让每趟运输都满载——于是整条路的吞吐被拉向理论上限。

- decode 优化 = 压缩访存（fp16 权重减半、KV 量化、更少权重加载/步），PagedAttention 的向量化与双缓冲都是”带宽侧”手段；

- prefill 优化 = 提升复用（分块注意力的大 tile、批量连续注意力），是”算力侧”手段；
- Continuous Batching = 让 GPU 始终工作在某一个区间的高点（batch 越大，GEMM 效率越高），本质是把多个请求的访存合并，抬高整体算术强度。

19.2 数值一致性是推理引擎的隐形地基

本项目多次教训表明：输出”看起来对”与”逐位对”之间隔着 kernel 路径、归约顺序、dtype 转换、写读次序等无数细节。投机解码 100% 接受率、前缀缓存共享、多模型对拍——全部建立在单一权威前向路径之上。工程启示：

- 同一算子的所有调用方共用同一 kernel（不做”快速路径”复制）；
- 状态（KV、hidden）只允许一个写入者；
- 参考实现与实现解耦（numpy/transformers 对拍而非自我对拍）。

19.3 显存、算力、延迟的不可能三角

6GB 消费卡上：fp16 换显存、KV 池预算换并发、前缀缓存换 prefill 时间、投机解码换 decode 吞吐、分块 prefill 换尾延迟。每一项优化都在三角上移动工作点而非消除约束。**推理引擎工程 = 在资源约束下显式选择工作点**，而好的架构（可插拔调度策略、可切换前缀缓存、可开关投机）让工作点选择成为运行时配置而非编译期宿命。

19.4 从零构建的价值

手写 12 类 kernel 的工程产出未必超过 cuBLAS/CUTLASS，但其价值在于：**每个优化决策都经过了”为什么”的检验**（双缓冲在 GEMM 上失败、在注意力上成功；Split-K 阈值 256；预采样融合……），这些第一手经验是使用黑盒库永远无法获得的。对学习者而言，这是比性能数字更宝贵的资产。

19.5 未来方向

- 树注意力/多候选投机（一次验证多条草稿路径）；
- EAGLE/Medusa 式免草稿模型投机；
- KV 量化（fp8/int8）与更长上下文的稀疏注意力；
- GPU 端采样与残差计算（当前为 CPU 热点）；
- 多卡张量并行（当前单卡）；
- 前缀缓存与投机解码的块级协同（避免污染共享块）。

A 项目文件结构速览

```
lightllm/
|-- include/lightllm/
|   |-- tensor.h / allocator.h      张量与显存抽象
|   |-- ops/                        算子声明 + dispatch 宏
```

```
|  |-- kv_cache/                PagedAttention / BlockAllocator / PrefixCache
|  |-- engine/                  Scheduler / EngineServer / BatchMainLoop /
|  |                             DraftEngine / SpeculativeDecoder
|  |-- model/                   ModelConfig / ModelLoader / TransformerBlock
|  |-- tokenizer/               GPT-2 BPE
|  `-- server/                  HTTP Server
|-- src/                        实现 (.cpp / .cu)
|-- tests/                      19 个测试程序
|-- tools/                      基准与调试工具
|-- docs/                       LaTeX 手册与报告
`-- third_party/xgrammar/       结构化输出依赖 (submodule)
```

B 关键常量与配置

常量/配置	默认值	说明
BLOCK_SIZE	16	物理块 token 数
SPLIT_K / 阈值	4 / 256	decode 长序列切分
MAX_SHMEM	48 KB	共享内存上限 (sm_75 默认)
chunk_size	16	prefill 分块大小
max_batch_tokens	256	每步 token 预算
LIGHTLLM_PREFIX_CACHE	hash	hash / radix
num_draft_tokens	4	投机草稿数 k
GEMM tile	64×64, TK=8	Vectorized/DoubleBuf

C 类比速查表

全文的” 形象类比” 汇总如下，便于快速回忆各技术点的直觉：

技术	生活类比	一句话直觉
Tensor 所有权	借书证	改书名可以，严禁先还书
GEMM 分块	超市理货	把货搬上柜台再卖，少跑仓库
GEMM 双缓冲	双柜台换班	换班搬货太贵时，不如单柜台
PagedAttention	图书馆格子	按取书单取散页，不誊抄全稿
在线 softmax	滚动记账	只记两个数，永不回头
散射写 KV	邮局分拣	一次倒进全部信箱
前缀缓存	乐高展架	拼过的龙头直接搬来用
Radix 分裂	拆积木插分支	从中间拆开，旧段照常共享
Continuous Batching	自助餐流水线	每勺都满，谁到给谁盛
DecodeFirst	急诊分诊	在跑请求优先，新人排队
容量预过滤	检查乐器够不够	不够这遍先不排
采样	转盘抽奖	温度调面积，掩码抠掉非法项
fp16 混合精度	两本账	货单可简写，账目必须精确
投机解码	老师批改作业	小助手先写，老师一次批
输出分布守恒	卷面统计等价	随机押题 + 按比批改 = 亲笔
结构化输出	填答题卡	每格只许填允许的选项
BPE 分词	乐高拼字	按手册反复拼相邻块